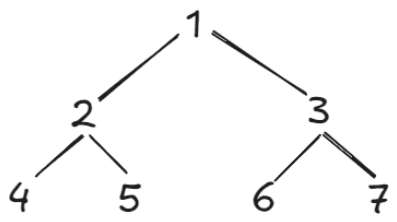
Model Examen – Seria 13
Rezolvari

1 AVL Trees

Cerinta: Care e numarul maxim de noduri al unui arbore AVL de inaltime 2? Presupunem ca inaltimea unui arbore cu un nod este 0.

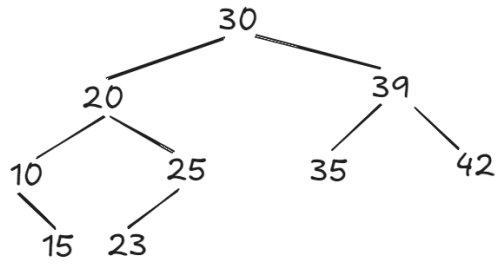
Numarul maxim este 7.



2 Tree walks

Cerinta: Traversarea in postordine a unui arbore binar de cautare este **15, 10, 23, 25, 20, 35, 42, 39, 30**. Care din urmatoarele este traversarea in preordine pentru acelasi arbore?

Pentru a obtine arborele din traversarea in postordine: ultimul nod este radacina; impartim restul nodurilor in multimea nodurilor subarborelui stang si multimea nodurilor subarborelui drept, iar apoi aplicam acest proces in mod recursiv. Astfel, obtinem urmatorul arbore si traversarea in preordine:



Postordine: 15 10 23 25 20 35 42 39
Preordine: 30 20 10 15 25 23 39 35 42

3 Binary Search Trees

Cerinta: Cand facem o stergere prin copiere (delete by copy) intr-un arbore binar de cautare, trebuie intotdeauna sa:

a. copiem valoarea stearsa in nodul succesor.

b. copiem valoarea din nodul succesor in nodul cu valoarea stearsa.

c. cautam nodul succesor al valorii sterse.

d. raspunsurile nu sunt corecte.

Stergerea prin copiere se refera la procedura de stergere a unui nod cu doi copii. In acest caz, se gaseste succesorul nodului si se copiaza valoare succesorului in nodul care a fost sters (dar nu se copiaza valoarea nodului in succesor). Raspunsurile corecte sunt **(b)** si **(c)**.

4 Binary Search Trees

Cerinta: Urmatoarele numere sunt inserate succesiv intr-un arbore binar de cautare gol: **10, 2, 3, 5, 15, 12, 16**, fara alte ajustari. Care din urmatoarele afirmatii sunt adevarate?

a. arborele este un arbore AVL.

b. nodurile pot fi colorate cu rosu/negru, astfel incat sa obtinem un red-black tree.

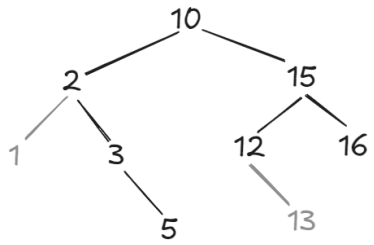
c. daca il inseram ulterior pe 13, parintele sau este 12.

d. daca il inseram ulterior pe 1, arborele va fi un arbore AVL.

Raspunsurile corect sunt **(c)** si **(d)**. Celelalte subpuncte sunt gresite, deoarece:

a. nodul 2 nu este echilibrat (diferenta dintre inaltimile subarborilor sai este de 2 noduri).

b. este imposibil (subarborele stang va fi mereu gresit).



5 Hash Tables

Cerinta: Care dintre urmatoarele afirmatii privind incarcarea (load) al unei tabele de dispersie (hash table) sunt adevarate?

a. este un numar intre 0 si 1.

b. este un numar intre 0 si 1 daca folosim open addressing.

c. este un numar intre 0 si 1 daca folosim chaining.

d. se dubleaza daca numarul de elemente se dubleaza.

Luam fiecare varianta pe rand:

a. gresit (poate fi intre 0 si 1, dar nu mereu).

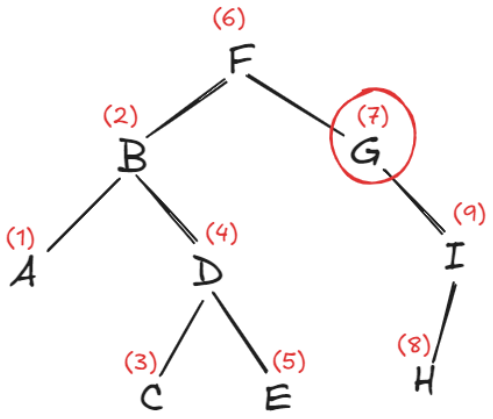
b. corect, pentru ca numarul maxim de elemente (in cazul asta) este dat de numarul total de slot-uri.

c. gresit, deoarece listele inlantuite de la fiecare slot pot avea mai multe elemente decat slot-uri.

d. consideram **m** numarul de elemente din tabela, iar **n** dimensiunea tabelei. Daca se dubleaza numarul de elemente, obtinem $\frac{2m}{n} = 2\frac{m}{n}$. Astfel, se dubleaza incarcarea $\rightarrow$ raspuns corect.

6 Binary Search Trees

Cerinta: Presupunand ca arborele de mai jos este unul de cautare, unde se afla al saptelea element?



7 Convex Hulls

Cerinta: Cand calculam infasuratoarea convexa a n puncte cu algoritmul Jarvis' March, trebuie sa:

a. folosim o stiva.

b. gasim punctul cu cea mai mica valoare a coordonatei O_y.

c. sortam punctele dupa unghiul polar.

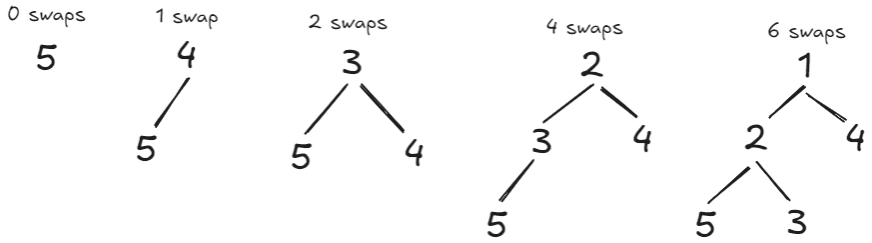
d. raspunsurile nu sunt corecte.

Raspunsurile **(a)** si **(c)** sunt gresite, deoarece nu folosim sortari sau stive. Punctul **(b)** ar putea fi corect (dar e cam subiectiv) - primul pas in algoritm este gasirea unui punct care sigur va face parte din infasuratoarea convexa; acest punct ar putea fi cel mai din stanga, sau cel mai de jos, etc. Astfel, este corect sa gasim punctul cu cea mai mica valoare a coordonatei O_y, dar nu este neaparat necesar (ca am putea sa luam alt punct).

8 Heaps

Cerinta: Presupunem ca inseram elementele **5, 4, 3, 2, 1** intr-un min-heap, adaugand succesiv fiecare element la coada heap-ului si apeland de fiecare data **heapify()**. Cate operatii **swap** face acest algoritm in total (un swap este o operatie in care un element si parintele lui in heap sunt interschimbate)?

Au loc 6 swap-uri.



9 Hash Tables

Cerinta: Consideram elementele **4322, 1334, 1471, 9679, 1989, 6171, 6173, 4198** si functia hash **h(x) = x mod 11**. Care din urmatoarele afirmatii sunt adevarate?

a. 1471 si 6171 produc o coliziune.

b. 9679, 1989 si 4198 se mapeaza la aceeasi valoare.

c. toate elementele sunt mapate la aceeasi valoare.

d. fiecare element este mapat la o valoare distincta.

Trecem prin fiecare varianta:

a. **gresit**, deoarece **h(1471) = 8** si **h(6171) = 0**.

b. **gresit**, deoarece **h(9679) = 10** si **h(1989) = 9**.

c. **gresit** (la subpunctele anterioare avem elemente mapate pe valori distincte).

d. **gresit** (avem coliziune intre **h(4322) = 10** si **h(9679) = 10**).

10 Tree walks

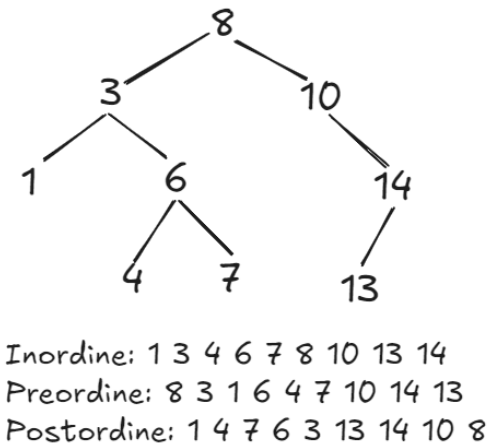
Cerinta: Care din urmatoarele secvente **sunt** una din traversarile in preordine/inordine/postordine ale arborelui de mai jos?

a. 8, 3, 10, 1, 6, 14, 4, 7, 13.

b. 8, 3, 1, 4, 6, 7, 10, 13, 14.

c. 1, 4, 7, 6, 3, 13, 14, 10, 8.

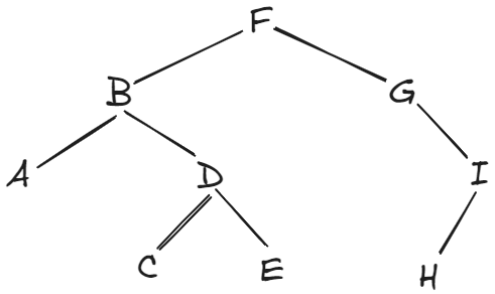
d. raspunsurile nu sunt corecte.



Punctul **(c)** este corect.

11 Red-Black Trees

Cerinta: Ce noduri sunt in mod sigur colorate cu rosu in arborele de mai jos daca trebuie sa fie un arbore red-black? Nodurile sentinela nu sunt reprezentate, iar valorile numerice sunt omise.



Raspuns: arborele e gresit si nu poate fi reprezentat ca un red-black tree. Daca ignoram subarborele stang si incercam sa coloram doar subarborele drept (F, G, I, H), este efectiv imposibil (din cauza nodului sentinela de la G).

12 Dynamic Sets

Cerinta: Vrem sa implementam urmatoarele operatii pe o multime dinamica de elemente S: insert(X), delete(X), search(X), average(X). Vrem ca operatiile sa aiba complexitate $O(\log n)$ worst-case. Care este cea mai simpla structura de date de la care putem porni (eventual modificand-o)?

a. Vector.

b. Hash Table.

c. Splay Tree.

d. AVL Tree.

e. Order Statistics Tree.

Trecem prin fiecare varianta:

a. **gresit:** vectorii sunt inutili pentru operatii de genul in $O(\log n)$, efectiv nu au treaba cu asta (average-ul, de exemplu, evident ca este $O(n)$ si nu poate fi imbunatatit in mod trivial).

b. **gresit:** hash table-uile nu au treaba cu operatia de search sau average, nu pentru asta au fost facute.

c. **gresit:** splay tree-ul are alte scopuri, cum poti imbunatati average()??

d. **gresit:** avl tree-ul are alte scopuri, cum poti imbunatati operatia average()??

e. **corect:** pentru asta folosim order statistics trees.

13 Trees

Cerinta: Care arbori au o garantie de complexitate $O(\log n)$ worst-case pentru complexitatea operatiilor?

a. AVL.

b. Splay.

c. Red-Black.

d. Order Statistics.

e. raspunsurile nu sunt corecte.

Arborii AVL si arborii red-black, deoarece acestia sunt echilibrati si au o limita a inaltimii marginita de $\log n$. Este subiectiv daca includem si order statistics, deoarece depinde care este tipul de arbore folosit la baza (putem combina order statistics cu AVL). Astfel, raspunsurile corecte sunt **(a)** si **(c)** (probabil nu includem **(d)**.

14 Hash Tables

Cerinta: Consideram o schema de hashing cu open addressing, care foloseste metoda de double hashing cu o tabela de marime 7, prima functie $h_1(k) = k \texttt{ mod } 7$, iar a doua functie $h_2(k) = 1 + (k \texttt{ mod } 5)$. Inseram elementele **27, 43, 692, 72** succesiv in tabela. Care celule sunt goale la sfarsitul procesului?

- **insert(27):** $h_1(27) = 6$.
- **insert(43):** $h_1(43) = 1$.
- **insert(692):** $h_1(692) = 6$, care produce o coliziune. Aplicam cealalta functie: $h_2(692) = 1 + 2 = 3$. Aplicam formula $(h_1(x) + i * h_2(x)) \texttt{ mod } 7$: pentru **i=1**, obtinem $(6 + 1 * 3) \texttt{ mod } 7 = 2$.
- **insert(72):** $h_1(72) = 2$, care produce o coliziune. Aplicam cealalta functie: $h_2(72) = 1 + 2 = 3$. Aplicam formula; pentru **i=1** avem $(2 + 1 * 3) \texttt{ mod } 7 = 5$.
- **Raspuns final:** celulele ocupate sunt **1, 2, 5, 6**, iar cele libere sunt **0, 3, 4**.

15 Trees

Cerinta: In care arbori de cautare trebuie, in general, sa efectuam rotatii?

a. AVL.

b. Splay.

c. Red-Black.

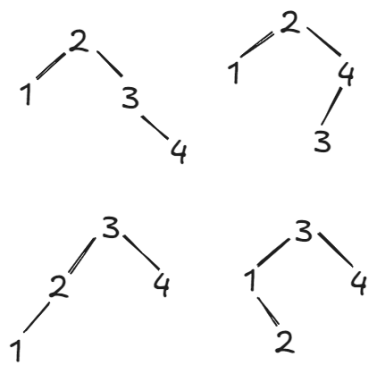
d. Order Statistics.

Raspunsurile corecte sunt **(a)**, **(b)** si **(c)**; totusi, punctul **(d)** este subiectiv, deoarece order statistics tree poate fi implementat cu un BST, sau cu un AVL, etc. Probabil nu trebuie inclus ca si raspuns corect, dar e subiectiv.

16 AVL Trees

Cerinta: Vrem sa reprezentam multimea **S = {1, 2, 3, 4}** cu un arbore AVL. In cate moduri putem face acest lucru?

In 4 moduri.



17 Interval Trees

Cerinta: De ce adaugam campul tree-size la un arbore interval?

a. pentru a calcula in mod eficient numarul de noduri din arbore.

b. pentru a face operatiile sa ruleze in timp $O(\log n)$.

c. pentru ca putem calcula tree-size in $O(1)$ din valorile campului tree-size al nodurilor copii.

d. nu il adaugam.

Raspunsul **(d)** este corect.

18 AVL Trees

Cerinta: Consideram urmatorul arbore AVL (valorile verzi reprezinta factorul de echilibru al nodului). Care din noduri poate fi ultimul adaugat (presupunem ca adaugarea nu a produs rotatii)?

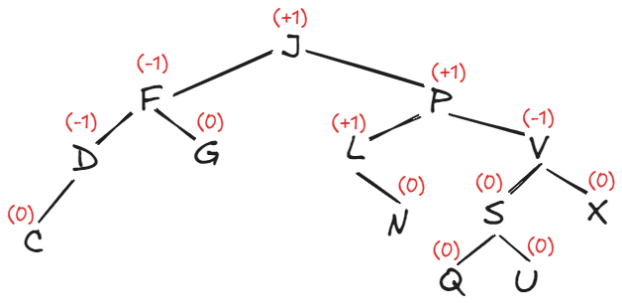
a. Nodul C.

b. Nodul G.

c. Nodul N.

d. Nodul U.

e. Raspunsurile nu sunt corecte.



Nodul **U**, adica punctul **(d)**. Pentru celelalte variante, daca scoatem nodul se strica factorul de echilibru.

19 Minimum Spanning Trees

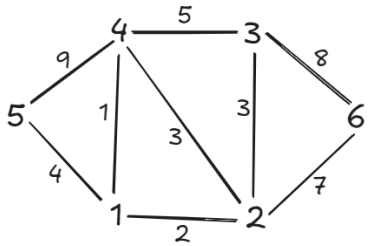
Cerinta: In graful de mai jos aplicam algoritmul lui Kruskal. Care din urmatoarele afirmatii **nu** pot fi adevarate?

a. Muchia (1,4) este prima adaugata in MST.

b. Muchia (1,2) este a doua adaugata in MST.

c. Muchia (2,4) este cea de-a 3-a adaugata in MST.

d. Muchia (2,3) este cea de-a 3-a adaugata in MST.



Raspunsurile corecte sunt **(a)**, **(b)** si **(d)**. Raspunsul **(c)** este gresit, deoarece ar forma un ciclu.

20 Queues

Cerinta: Care din urmatorii algoritmi folosesc o coada in implementarea standard?

a. Graham's Scan.

b. Depth-First Search.

c. Breadth-First Search.

d. Raspunsurile nu sunt corecte.

Raspunsul corect este **(c)** (deoarece Graham's Scan foloseste o stiva, iar DFS este recursiv - ca o stiva).

21 Skip Lists

Cerinta: La inserarea unui element intr-un skip list pe nivelul maxim 2 (nivelurile incep cu 0), in cate noduri (incluzand nodul nou inserat) trebuie sa fie modificati pointerii?

Cel inserat, cel din stanga si cel din dreapta (**3** noduri).

22 B-Trees

Cerinta: Cea mai mare cheie intr-un B-Tree cu mai mult de un nod poate fi situata:

a. In radacina, dar nu altundeva.

b. Intr-o frunza, dar nu altundeva.

c. Si intr-o radacina si intr-o frunza, dar nu altundeva.

d. Mai exista si alte alternative neluate in calcul.

Sa presupunem ca avem doar doua noduri (radacina si un copil). Cea mai mare cheie ar putea fi in radacina (daca al doilea nod este copil stang) sau intr-o frunza (daca e copil drept). In restul cazurilor, ar trebui sa se gaseasca in frunza cea mai din dreapta (pentru ca acolo e cea mai mare valoare), deci raspunsul corect e **(c)**.

23 Linked Lists

Cerinta: Care este numarul de pointeri (incluzand pointerul head) intr-o lista simplu inlantuita?

Avem pointerul head, pointerul de la ultimul nod spre NULL si inca n-1 pointeri; in total sunt $\mathbf{n + 1}$ pointeri.

24 Heaps

Cerinta: O secventa de intregi se numeste **heapable** daca elementele ei pot fi inserate succesiv drept ultimul element in vectorul care reprezinta min-heap-ul, fara a fi nevoie sa chemam functia heapify. Care din urmatoarele secvente sunt heapable?

a. 2

b. 2 4 5 3

c. 2 4 3 5

d. 5 2 3 4

Parerea mea: cred ca, de exemplu, pentru secventa **2 4 3 5** iti spune ca inserezi mai intai 2 ca ultimul element, apoi 4 ca ultimul element, etc, si obtinem fix aceeasi secventa (deci nu stiu de ce e asa de lunga cerinta; sper sa nu gresesc). In acest caz, singura secventa care respecta proprietatea de min-heap este cea de la punctul **(c)**.

25 Geometry

Cerinta: Cand rulez algoritmul care testeaza intersectia segmentelor:

a. folosesc un arbore splay.

b. testez segmente pentru intersectie si cand adaug un capat si cand sterg unul.

c. obtin un algoritm cu complexitate $O(n \log n)$.

d. folosesc drept primitiva un algoritm de sortare.

Trecem prin fiecare varianta:

a. **gresit:** nu se folosesc arbori splay, ci arbori red-black.

b. **corect:** asa este pseudocodul din curs.

c. **corect:** asta este complexitatea din curs.

d. **corect:** primul pas este sortarea punctelor.